

Query Plans, Measurements, and Index Design

Read the work the database chose, test one hypothesis, and keep only justified access paths.

Performance begins with a workload question

Faster for which query shape, data distribution, scale, and objective?

WORKLOAD

Predicate, join, sort, grouping, frequency, and concurrency.

OBJECTIVE

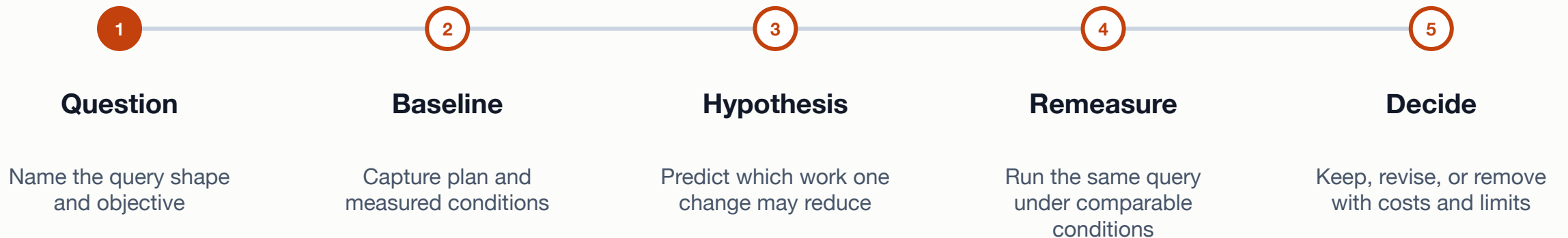
Latency, throughput, resource use, or predictable tail behavior.

EVIDENCE

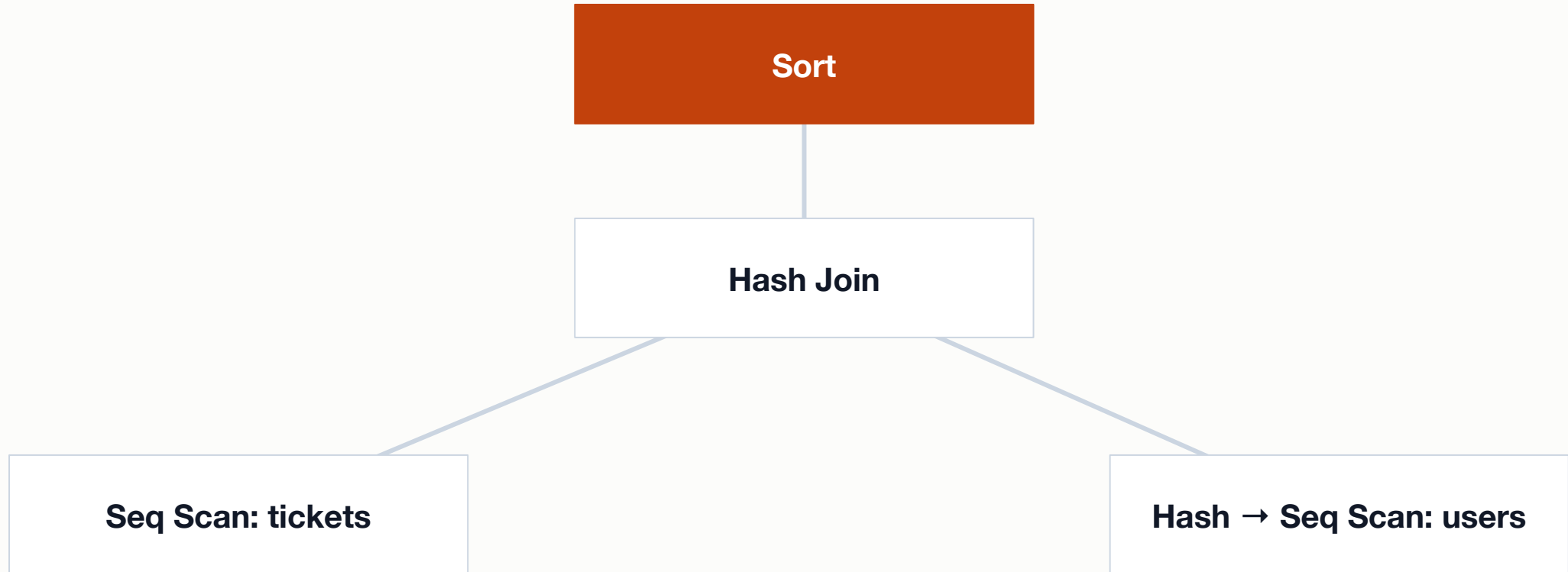
Plan nodes, rows, loops, buffers, timing, and repeatable conditions.

A controlled experiment changes one cause at a time

The decision follows the evidence; it does not precede the baseline.



Read a plan from the leaves toward the final result



Child nodes produce rows for their parent; indentation expresses the execution tree.

EXPLAIN ANALYZE adds actual execution evidence

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, opened_at, subject
FROM performance_lab.tickets
WHERE category = 'streetlight'
      AND status IN ('new','open','in_progress')
ORDER BY opened_at DESC
LIMIT 25;
```

Safety first

- ANALYZE runs the query; use safe SELECT fixtures.
- BUFFERS reports cache/block activity in context.
- Preserve the exact query and plan text.

Estimates guide planning; actuals reveal execution

Estimated evidence

- Cost range and estimated row count
- Derived from statistics and planner models
- Used to choose among candidate plans

Actual evidence

- Measured rows, loops, and timing for this run
- Affected by parameters, cache, load, and environment
- Large estimate errors can point to stale or insufficient statistics

Sequential and index access are choices, not grades

Sequential scan

- Reads table blocks broadly
- Can be cheapest for small tables or large result fractions
- Uses efficient sequential access and avoids index lookup overhead

Index-based access

- Navigates an index to candidate tuple locations
- Can reduce work for selective predicates or useful order
- May still perform heap visits and random access

Lab 1: Read the plan before changing anything

Annotate one PostgreSQL plan and identify the dominant work, estimate quality, and one testable hypothesis.

- Record the exact query, result grain, table scale, and baseline conditions.
- Read leaf-to-root and annotate rows, loops, buffers, and any sort or join.
- State one hypothesis about a single change without applying it yet.

SUBMIT ONE THING / one annotated baseline plan

An index is justified by the workload it improves and the cost it adds

One before-and-after experiment is stronger than a folder of unused indexes.

Compound index order should reflect the query shape

Query component	Example	Index design implication	Caution
Equality filter	category = 'streetlight'	Often useful early in a B-tree key	Low selectivity may limit value
Set/equality filter	status IN (...)	May follow another equality key	Distribution and combinations matter
Order/range	opened_at DESC	Can provide order after leading keys	Range can limit later-key use
Returned fields	ticket_id, subject	INCLUDE may reduce heap visits	Wider index increases cost

Compare the mechanism, not only the elapsed-time number

Evidence	Baseline	With candidate index	Interpretation
Access	sequential scan	index or bitmap path	Did the candidate target the predicate?
Sort	explicit sort	removed or smaller	Did key order support requested order?
Rows / buffers	broad work	reduced or unchanged	Where did work move?
Runtime	repeatable range	repeatable range	Is change meaningful under conditions?

Keeping an index creates continuing operational work

- 1 Measure benefit for the named read workload and check neighboring queries.

- 2 Account for insert, update, delete, storage, cache, and build costs.

- 3 Name ownership, monitoring, statistics, vacuum, and maintenance expectations.

- 4 State fixture and cache limits; schedule remeasurement when scale or workload changes.

Lab 2: Test one index hypothesis

Create one candidate index, remeasure the same query, and decide whether its benefit justifies its costs.

- Predict the access, sort, row, or buffer change the index should cause.
- Create only that candidate and rerun the identical query under comparable conditions.
- Compare plans and recommend keep or drop with one cost and one limitation.

SUBMIT ONE THING / one before/after index recommendation

Performance claims require a baseline, mechanism, and tradeoff

Read the plan, test one hypothesis, preserve correctness, and pay for only useful indexes.

- Where did the plan perform its work?

- What changed after the index?

- Which ongoing cost accompanies the benefit?